周报1

本周已刷题目

洛谷：入门 × 7 普及 × 1

力扣：简单 × 3 中等 × 2

- 数组
    1. P1427 小鱼的数字游戏
    2. P5727 【深基5.例3】冰雹猜想
    3. NOIP 2005 普及组] 校门外的树
    4. P5728 【深基5.例5】旗鼓相当的对手
    5. Q2. 有多少小于当前数字的数字
    6. P5729 【深基5.例7】工艺品制作
    7. P2550 AHOI2001\彩票摇奖
    8. leetcode162. 寻找峰值
- 模拟
    1. NOIP 2003 普及组] 乒乓球 - 洛谷
    2. NOIP 2015 普及组扫雷游戏
- 链表
    1. leetcode206-反转链表
    2. leetcode21.合并两个有序列表
    3. leetcode2. 两数相加

学习知识点

1. c语言计算数组长度，通过`sizeof`测量数组内存的长度和存入数据类型作商得到
2. c语言`<stdlib.h>`头文件有生成伪随机数的函数，原理是线性同余方程，一般用`<time.h>`库中的`time()`函数获取时间戳，作为生成随机数的种子，尽可能让每次生成的随机数不一样。

```c
#include<stdlib.h>
#include<time.h>

main()
{
    srand(time(NULL));
    int a = rand() % 100 + 1; // 生成[1,100]的随机整数
    return 0;
}
```

   100表示随机数的范围宽度，1是区间左端点

3. 差分数组可以用来对连续区间进行操作。

   差分是前缀和的逆运算

```
int a[];
for(int i = 1; i <= n; i++)
{
    s[i] = a[i] - a[i - 1]; //s 差分数组
}
// 前缀和还原
for(int i = 1; i <= n; i++)
{
    s[i] += s[i - 1];    //前缀和还原（若中间没做任何操作，则 s[i] 与 a[i] 相等）。
}
```

对区间[a,b]进行操作

```
dif[a] +x, dif[b + 1] - x; // 对区间[a,b]加x
```

4. c语言中<stdlib.h>头文件中有绝对值函数abs()
5. 用指针遍历二维数组

```
int arr[5][5] =
{
    {1,2,3,4,5},
    {6,7,8,9,10},
    {11,12,13,14,15},
    {16,17,18,19,20},
    {21,22,23,24,25}
};
// 用指针打印二维数组
int(*p)[5] = arr; // 获取二维数组指针，数据类型是二维数组中第一个元素的类型
for(int i = 0; i < 5; i++)
{
    for(int j = 0; j < 5; j++)
    {
        printf("%d ",*(*(p + i) + j));
    }
    printf("\n");
}
```

6. python中导入sys库，可用其中的标准输入文件stdin和标准输出文件stdout对批量的输入输出数据进行操作。
7. 链表是一种跳转结构的数据结构，它在内存里不是连续的。其中每个节点中含有当前节点的值和下一节点的地址，尾部节点指向的内存地址为null.

刷题笔记

- 数组
  - P5727 【深基5.例3】冰雹猜想

    先把差分数组初始化为0，遍历要把树移除的区间，在差分数组左端点索引处加1，右端点索引后一位处减1。最后用求前缀和的方式还原差分数组，再遍历统计数组中0的个数，及剩余树木数量。
  - Q2. 有多少小于当前数字的数字

用一个记录数组统计每个数出现的次数，数组索引的数字n就代表原数组记录的数字。

最后用列表推导式构造一个结果数组，每一项计入原数组n在记录数组中的前n-1项和，表示原数组比n小的数出现次数和。

- P5729 【深基5.例7】工艺品制作

  通过对三维数组中的一维数组差分提高效率。

  先对数组初始化为0，用两层for循环遍历三位数组中的二维数组，对二维数组中的一维数组进行连续区间差分操作，最后用三层for循环遍历进行前缀和还原，并统计不变的元素。

  若数据量更大，可用三维差分数组执行，对要操作的区域进行8点标记，最后还原时依次从$x->y->z$分别前缀和还原。

- 162. 寻找峰值

  题目要求时间复杂度为$O(logn)$，可用二分查找。

  当数组长为1，$nums[0]$就是峰值，数组长度>=2时，依次判断首末位置是否是峰值，判断结束后，峰值一定在$[1,n-2]$位置上。

  用二分法搜索区间：

  - 求中点m，
  - 判断m是否为峰值，m是峰值则返回m，
  - 若$nums[m-1]>nums[m]$，则$[left,m-1]]$内有峰值，对区间二分；
  - 若$nums[left]<nums[m]$，则$[m+1,right]]$内有峰值，对区间二分。

- leetcode21.合并两个有序列表

  定义哨兵节点，可以简化边界处理

  - 定义一个临时结构体head，其 next 初始化为 NULL，避免处理**空链表**、**头节点选择**等边界问题，简化代码。
  - pre指针指向已合并列表的最后一个节点，用$pre->next$来拼接节点.
  - 只要 `list1` 和 `list2` 都不为空，就比较两者当前节点的 `val`，将较小的节点拼接到 `pre->next`，并移动对应链表的指针。并把`pre`指针指向拼接好的最后一个节点。
  - 循环结束后，其中一个链表已遍历完，直接将剩余链表拼接到合并链表末尾。

- leetcode2. 两数相加

  用链表模拟两数相加

  - 定义临时结构体`head`， 仅作为占位符，不存储有效数值，所有有效节点都挂在 `head.next` 下，避免头节点赋值错误。
  - 所有新节点的`next`都赋值为`NULL`，避免尾部出现空链表.
  - 只要有一个链表不为空或进位`carry`上有大于0的值，就进行循环.
  - 建立新节点，用`malloc()`动态分配内存，避免循环结束后栈内存释放，导致指针指向空.

附件

数组

1. P1427 小鱼的数字游戏

   倒序打印一串数组

```
#include <stdio.h>

```

```c
int main()
{
    int an[100];
    int i = 0;
    while(1)
    {
        int a;
        scanf("%d",&a);
        if(!a)
        {
            break;
        }
        an[i] = a;
        i++;
    }
    for(int j = i - 1; j >= 0;j--)
    {
        printf("%d ",an[j]);
    }
    return 0;
}
```

2. P5727 【深基5.例3】冰雹猜想

角古猜想，用数组存储数据变化并逆序打印。

```c
#include <stdio.h>

int main()
{
    int a;
    scanf("%d",&a);
    int ar[10000];
    int i = 0;
    while(1)
    {
        ar[i] = a;
        i++;
        if(a == 1)
        {
            break;
        }
        else if(a % 2 == 1)
        {
            a = a * 3 + 1;
        }
        else
        {
            a /= 2;
        }
    }
    for(int j = i - 1; j >= 0; j--)
    {
```

```
        printf("%d ",ar[j]);
    }
    return 0;
}
```

3. NOIP 2005 普及组] 校门外的树

```
#include <stdio.h>

int main()
{
    int met;
    int itvs;
    scanf("%d %d",&met,&itvs);
    // 用差分数组存储对连续区间的操作
    int dif[met + 2]; // 防止索引越界
    for(int i = 0; i < met + 2; i++)
    {
        dif[i] = 0;
    }

    int lef, rig;
    for(int i = 0; i < itvs; i++)
    {
        scanf("%d %d",&lef,&rig);
        dif[lef]++, dif[rig + 1]--;
    }

    // 还原数组
    for(int i = 1; i <= met; i++)
    {
        dif[i] += dif[i - 1];
    }
    // 统计
    int count = 0;
    for(int i = 0; i <= met; i++)
    {
        if(!dif[i])
        {
            count++;
        }
    }
    printf("%d",count);
    return 0;
}
```

4. P5728 【深基5.例5】旗鼓相当的对手

```
#include <stdio.h>
#include <stdlib.h>
```

```c

int main()
{
    int row;
    scanf("%d",&row);
    int a[row][4];
    for(int i = 0; i < row; i++)
    {
        scanf("%d %d %d",&a[i][0],&a[i][1],&a[i][2]);
        a[i][3] = a[i][0] + a[i][1] + a[i][2];
    }

    int par = 0;
    for(int i = 0; i < row - 1; i++)
    {
        for(int j = i + 1; j < row; j++)
        {
            int y = abs(a[i][0] - a[j][0]);
            int m = abs(a[i][1] - a[j][1]);
            int e = abs(a[i][2] - a[j][2]);
            int sum = abs(a[i][3] - a[j][3]);
            if(y <= 5 && m <= 5 && e <= 5 && sum <= 10)
            {
                par++;
            }
        }
    }
    printf("%d",par);
    return 0;
}
```

5. Q2. 有多少小于当前数字的数字

```python
class Solution:
    def smallerNumbersThanCurrent(self, nums: List[int]) -> List[int]:
        # 桶思想：用桶记录每个数出现次数
        a = [0] * 101 # 记录每个数出现次数
        for i in nums:
            a[i] += 1

        # nums数组中比n小的数就是a得前n-1项
        return [sum(a[0:n]) for n in nums] # 计算nums中元素在a表作为索引的前缀和
```

6. P5729 【深基5.例7】工艺品制作

```c
#include <stdio.h>

int main()
{
    // 初始化三维数组
```

```
    int w,x,h,q;
    scanf("%d %d %d",&w,&x,&h);
    int sqr[w + 2][x + 2][h + 2]; // 防止索引越界
    for(int i = 0; i <= w; i++)
    {
        for(int j = 0; j <= x; j++)
        {
            for(int k = 0; k <= h; k++)
            {
                sqr[i][j][k] = 0;
            }
        }
    }

    // 输入q行切割区间
    scanf("%d",&q);
    for(int r = 0; r < q; r++)
    {
        int i0,j0,k0,i1,j1,k1;
        scanf("%d %d %d %d %d %d",&i0,&j0,&k0,&i1,&j1,&k1);
        for(int i = i0; i <= i1; i++)
        {
            for(int j = j0; j <= j1; j++)
            {
                sqr[i][j][k0]++,sqr[i][j][k1 + 1]--;
            }
        }
    }

    // 统计表示立方体三维数组中为0的元素
    int rem = 0;
    for(int i = 1; i <= w; i++)
    {
        for(int j = 1; j <= x; j++)
        {
            for(int k = 1; k <= h; k++)
            {
                sqr[i][j][k] += sqr[i][j][k - 1];
                if(!sqr[i][j][k])
                {
                    rem++;
                }
            }
        }
    }
    printf("%d",rem);
    return 0;
}
```

7. P2550 AHOI2001\彩票摇奖

```
```

```c
#include <stdio.h>

int main()
{
    int paps;
    scanf("%d",&paps);
    int awd[7];
    scanf("%d %d %d %d %d %d
%d",&awd[0],&awd[1],&awd[2],&awd[3],&awd[4],&awd[5],&awd[6]);

    int rcd[7] = {0};
    // 外循环为小明买的彩票数量
    for(int t = 0; t < paps; t++)
    {
        int tic[7];
        int count = 0;
        for(int i = 0; i < 7; i++)
        {
            scanf("%d",&tic[i]);
            // 对输入的号码和中奖号码比对
            for(int j = 0; j < 7; j++)
            {
                if(tic[i] == awd[j])
                {
                    count++;
                    break;
                }
            }
        }
        switch(count)
        {
            case 7:
                rcd[0]++;
                break;
            case 6:
                rcd[1]++;
                break;
            case 5:
                rcd[2]++;
                break;
            case 4:
                rcd[3]++;
                break;
            case 3:
                rcd[4]++;
                break;
            case 2:
                rcd[5]++;
                break;
            case 1:
                rcd[6]++;
                break;
            default:
```

```
                break;
            }
        }

        for(int i = 0; i < 7; i++)
        {
            printf("%d ",rcd[i]);
        }
        return 0;
}
```

8. 162. 寻找峰值

```
int findPeakElement(int* nums, int numsSize)
{
    if(numsSize <= 1)
    {
        return 0;
    }
    // 若数组长度>=2,判断首末位置是否为峰值
    if(*nums > *(nums + 1))
    {
        return 0;
    }
    else if(*(nums + numsSize - 2) < *(nums + numsSize - 1))
    {
        return numsSize - 1;
    }
    // 此处数组长度>=3且[1,n-2]区间必存在峰值点
    int left = 1;
    int right = numsSize - 2;
    int m;
    while(left <= right)
    {
        m = left + (right - left) / 2;
        if(*(nums + m - 1) > *(nums + m))
        {
            right = m - 1;
        }
        else if(*(nums + m) < *(nums + m + 1))
        {
            left = m + 1;
        }
        else
        {
            return m;
        }
    }
    // 防止编译器报错，实际无法到达。
    return m;
}
```

模拟

1. NOIP 2003 普及组] 乒乓球 - 洛谷

```python
import sys

# 读取信息
sco = []
for line in sys.stdin:
    line = line.strip()
    if 'E' not in line:
        sco.append(line)
    else:
        ind = line.index('E')
        sco.append(line[:ind])
        break

sco = "".join(sco)
# 打印比分
def prt_sco(system):
    p1 = 0
    p2 = 0
    for i in range(len(sco)):
        if sco[i] == 'W':
            p1 += 1
        elif sco[i] == 'L':
            p2 += 1
        if (p1 >= system or p2 >= system) and abs(p1 - p2) >= 2:
            print(f"{p1}:{p2}")
            p1 = 0
            p2 = 0
    #if p1 > 0 or p2 > 0:
    print(f"{p1}:{p2}")

prt_sco(11)
print()
prt_sco(21)
```

2. NOIP 2015 普及组扫雷游戏

```python
graph = []
n, m = map(int, input().split()) # n:行数 m:列数
for i in range(n):
    graph.append(input())

dev = [(-1, -1), (-1 , 0), (-1, 1),
       (0, -1),           (0, 1),
       (1, -1), (1, 0), (1, 1)]
for i in range(n):
    for j in range(m):
```

```
        if graph[i][j] == '*':
            print('*', end="")
        elif graph[i][j] != '*':
            # 遍历周围8个格子，查找有无地雷
            count = 0
            for dx, dy in dev:
                x = i + dx
                y = j + dy
                if 0 <= x < n and 0 <= y < m:
                    if graph[x][y] == '*':
                        count += 1
            print(count, end="")
    print()
```

链表

1. leetcode206-反转链表

```
/**
 * Definition for singly-linked list.
 * public class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode() {}
 *     ListNode(int val) { this.val = val; }
 *     ListNode(int val, ListNode next) { this.val = val; this.next = next; }
 * }
 */
class Solution {
    public ListNode reverseList(ListNode head) {
        // 双指针pre记录上一节点，next记录下一节点
        ListNode pre = null;
        ListNode next = null;
        while(head != null){ // 走到链表末尾
            next = head.next; // 把原下一节点赋值到next指针
            head.next = pre; // 让新的下一节点变为指向原来上一节点
            pre = head; // pre指针记录当前节点
            head = next; // 跳转下一节点操作
        }
        return pre; // return the spot reversed ultimately
    }
}
```

2. leetcode21.合并两个有序列表

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
```

```
 * };
 */
struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2)
{
    struct ListNode head;
    head.next = NULL;
    struct ListNode* pre = &head; // pre 指针指向当前合并链表的最后一个节点
    while(list1 != NULL && list2 != NULL)
    {
        if(list1->val < list2->val)
        {
            pre->next = list1;
            list1 = list1->next;
        }
        else
        {
            pre->next = list2;
            list2 = list2->next;
        }
        pre = pre->next;
    }
    // 拼接剩余节点
    pre->next = list1 != NULL ? list1 : list2;
    // head.next指向合并后第一个有效节点的地址
    return head.next;
}
```

3. leetcode2. 两数相加

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */
struct ListNode* addTwoNumbers(struct ListNode* l1, struct ListNode* l2)
{
    // 哨兵节点head，不进行赋值
    struct ListNode head;
    head.next = NULL;
    // 指针pre指向当前相加的节点
    struct ListNode* pre = &head;
    int carry = 0; // 记录进位
    while(l1 != NULL || l2 != NULL || carry > 0)
    {
        int sum = (l1 == NULL ? 0 : l1->val)
        + (l2 == NULL ? 0 : l2->val)
        + carry;

        int cval = sum % 10;
        carry = sum / 10;

```

```
        // 建立新节点，用malloc动态分配内存，避免循环结束后栈内存释放，导致指针指向空
        struct ListNode* new = (struct ListNode*)malloc(sizeof(struct ListNode));
        new->val = cval;
        new->next = NULL;

        pre->next = new;
        pre = pre->next;
        l1 = l1 == NULL ? NULL : l1->next;
        l2 = l2 == NULL ? NULL : l2->next;
    }

    return head.next;
}
```

下周计划

学习c语言动态内存分配，学习一些简单数据结构和算法，并尝试完成相应代码。